

Implementierung von Anwendungssystemen

SoSe 2026, M.Sc. Robin Burchard



- Anwendungssystem:
 - Informationstechnologisches System für konkrete Anwendung
 - Beinhaltet:
 - Versch. Software (Frontend / Backend)
 - Sonst. IT-Infrastruktur (Hardware, Daten, Netzwerk etc.)
- Dieses Praktikum:
 - Entwicklung von „Individualsoftware“
 - Spezifische Anwendung wird vorgegeben (gleich)
 - Anforderungen müssen erarbeitet werden



- Für Leistungen rechtzeitig anmelden!
 - Sonst kein Anspruch auf Prüfung / Abnahme einer Studienleistung
 - Prüfungsamt akzeptiert keine Nachmeldungen mehr!
- Alle Abgaben und Präsentationen werden benotet

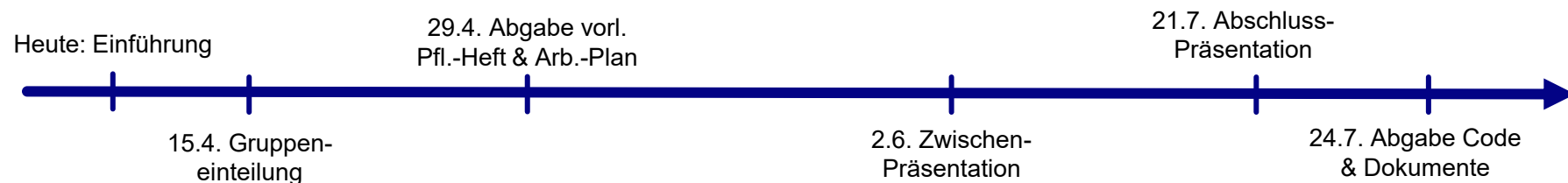
- **Kontakt** (Mail: robin.burchard@uni-siegen.de)
Praktikums-Mattermost: [Invite Link](#)
- **Termine:**
 - 29.04.26 23:59
 - Abgabe des (vorläufigen) Pflichtenheftes und Arbeitsplans (via Mattermost)
 - 02.06.26
 - Kurze Vorstellung des Zwischenstandes („Zwischenpräsentation“)
 - 21.07.26
 - Präsentation der finalen Software („Abschlusspräsentation“)
 - Wöchentlicher Termin (Di 8:30-10 Uhr c.t., H-C 7326), **KURZUPDATE***, im Team Besprechen, Fragen etc.
- **Programmierpraktikum, selbstständiges Arbeiten, Großteil der Arbeit nicht in Präsenz**
 - Zwischen-/Abschlusspräsentation werden in Präsenz vorgestellt, sonst keine generelle Anwesenheitspflicht*
 - **Bei Problemen unbedingt schnell reagieren und kommunizieren**

(* Siehe nächste Folie)





- Thema wird gegeben
- Ihr implementiert ein Anwendungssystem (Überraschung)
 - Backend (Server und Datenbankbindung)
 - Website und App werden erstellt
- *Wöchentliches Kurzupdate (Di. 8:30 PB-A 122)
 - Keine Folien o.Ä. benötigt
 - Pro Gruppe jede Woche mind. 1 Person anwesend
 - Kurz (< 3 min) erklären, was die neuesten Fortschritte sind





- Bearbeitung der Problemstellung in Gruppenarbeit mit 5 - 6 Personen
- Selbstständiges Arbeiten
 - Lernen durch Tutorials
 - Selbstorganisation in der Gruppe (Code, Test, Dokumentation, Benutzerhandbuch, Pflichtenheft, ...)
 - Am besten: Jede Woche treffen, absprechen wer was tut etc.
 - Bei Problemen:
 1. Google/Foren/Tutorials
 2. Eigene Gruppe
 3. Evtl. andere Gruppen via Mattermost („es gibt keine dummen Fragen“)
 4. Betreuer via Mattermost



- Zwischenpräsentation (10-15 Minuten)
 - Arbeitsplan
 - Anforderungen, Wünsche und Zielsetzungen
 - Milestones
 - Was ist erledigt, was ist noch zu tun
 - Mockups
 - Evtl. kurze Demo der Software
 - Evtl. Probleme/Schwierigkeiten
- Abschlusspräsentation (10-20 Minuten je Gruppe)
 - Demo der Software
 - Kurze Fragerunde im Anschluss
- Jeweils Anwesenheitspflicht, 8-12 Uhr einplanen!



- Selbstständiges Arbeiten
 - Selbstständige Einarbeitung in das Projekt
 - Selbstständige Erlernung der erforderlichen Kenntnisse
- Softwareentwicklung im Team
 - Organisation des Teams
 - Einhaltung von Terminen/Milestones
 - Simultane Bearbeitung des Projektes - Versionsverwaltung
 - Coding-Style und Dokumentation
- Arbeit an einem größeren Projekt
 - Umsetzung von unvollständiger Information in funktionierende Software
 - Programmiermuster: Model-View-Controller (MVC)



- Pflichtenheft
- Dokumentation
 - Wer hat was wann getan (kurze tabellarische Übersicht)
- Bedienungsanleitung
- Funktionierende Software (Code + Ressourcen)
 - Klassen und Funktionen sind dokumentiert, d.h. es ist jeweils eine kurze Beschreibung zu Zweck und Ein- und Ausgabewerten als Kommentarfeld im Code vorhanden
 - Fehlerfrei kompilierender Code (falls nötig step-by-step Anleitung zum Aufsetzen)
 - Abgabe als .zip oder Link zum Repo



- B.Sc Informatik
- Master Wirtschaftsinformatik
- ~~(B.Sc. Digital Biomedical and Health Sciences)~~
- Gemischte Gruppen – Gemischte Kompetenzen



- Gruppen mit je 5 - 6 Teilnehmern
- Gruppen werden eingeteilt (Formular Ausfüllen)
 - Nötig, um in eine Gruppe eingeteilt zu werden
 - Ehrlich sein so gut es geht
 - Präferenzen zu Gruppen möglich, aber nicht garantiert
 - Möglichst balancierte Gruppen (Studiengänge, Fähigkeiten)
- Mattermost Team beitreten:
 - Kommunikation in der Gruppe
 - Kommunikation zw. den Gruppen
 - Kommunikation mit dem Betreuer

Entwicklung einer Plattform zur Dokumentation von Zoologischen Sammlungen

Entwicklung einer Plattform zur Dokumentation von Zoologischen Sammlungen

- Grundidee:
 - Wissenschaftler bzw. Museen sammeln Tiere, z.B. Insekten wie Schmetterlinge, Käfer,
 - Jedes gesammelte Individuum wird inventarisiert
- Grundanforderungen müssen umgesetzt werden
- Viele Features möglich!



Idee einer wissenschaftlichen Sammlung:

Aufbewahren von individuellen Objekten, deren Herkunft am Objekt dokumentiert ist.

Praxis: Objekte sind nach vorgegebenen Kriterien gruppiert (Taxonomie/ Größe/ Gewicht/ Ansprüche an Umgebungsbedingungen...).

Zielsetzung: Ausstellung, Ausleihen, Forschung

Taxonomie: Art-Begriff, Taxon, zool. System (Hierarchie: Art U.-Fam Fam Ordnung Klasse)



Beispielhafte Fragen an eine Sammlungs-DB:

- wo finde ich ein bestimmtes Taxon [hierarchische Struktur der Taxa und Standortkenntnis in Sammlung]
- von welchen Fundorten habe ich welche Taxa oder auch Individuen [indiv. Fundortkenntnis]
- aus welchen geographischen Bereichen gibt es in der Sammlung Objekte [hierarchische Struktur der Fundorte]
- wie dokumentiere ich das Entleihen eines sammlungseigenen Objekts an Fremde?
- Wie dokumentiere ich das Ausleihen und Aufbewahren eines Objekts aus einer fremden Sammlung? (Die andere Partei nutzt nicht zwingend unser Anwendungssystem!)



- Website
 - (nicht eingeloggt)
 - Möglichkeit sich zu registrieren / einzuloggen
 - Kompletter Zugang zu öffentlich eingetragenen Sammlungen (Karte u. Liste):
 - **Karte**: Funde als Overlay, z.B. mit Icons nach Kategorie, Filtern!
 - (eingeloggt Nutzer/in)
 - Möglichkeit, Tiere in eigener Sammlung einzugeben:
 - SCOFGA (Taxonomie) Maske, mit Suche / Ausfüllung
 - Neue Arten (inkl. Taxonomie-Hierarchie) anlegen (müssen freigegeben werden)
 - Ort (Koordinaten oder über **Karte**), Datum, ...
 - Liste der eigenen Sammlung, öffentliche Sammlungen von anderen Nutzern
 - Detailseite für Funde:
 - Kreativität hier einfügen!
 - Import / Export (z.B. .csv) von Objekten der Sammlung
 - (eingeloggt, Moderator / verifizierter Nutzer), zusätzlich:
 - Neue Arten freischalten
 - Sammlungen moderieren (editieren, löschen)
 - (eingeloggt, Admin)
 - Moderation / Übersicht über alle Funde, Arten usw., generelle Nutzer und Inhaltsverwaltung (Vollzugriff)
 - Kann Nutzer verifizieren / befördern



- App (Android oder iOS oder beides)
 - Login für Nutzer, nach Login:
 - Teile der Funktion der Website
 - z.B. Suche nach Standort in Sammlung, Fundorten mit Karte etc.
 - Benachrichtigung, wenn Interaktion erfolgt ist, mit Einstellungsmöglichkeiten (deaktivieren), z.B.:
 - Neue öffentliche Eintragung von Bestimmtem Individuum
 - Leihfrist eines Individuums läuft bald ab
 - Nicht einfach ein reiner Webbrowser (darf aber HTML Elemente beinhalten)!
 - Muss keine Moderation / Administration ermöglichen
 - Darf aber natürlich



- Nutzer/in (eingeloggt)
 - KEIN ZUGANG zu Admin Funktionen
 - Registrierung/Identifikation mit Kennung/Nutzername
 - Kann eigene Sammlung erstellen / bearbeiten / öffentlich machen
- Moderator
 - Kann Arten anlegen / freigeben
 - Kann Nutzer sperren / freigeben



- Administrator
 - Nutzer mit Vollzugriff
 - Kann alle Nutzer
 - Erstellen, bearbeiten, sperren und löschen
 - Kann Admin-Berechtigungen zuweisen und entfernen
 - Kann alle Arten von Daten
 - Eingeben, bearbeiten und löschen
 - ➔ Hat überall Zugriff, kann alles, sieht alles, darf alles.



- Überlegt gut:
 - Wie wird die Hierarchie umgesetzt? (Geographisch und Taxonomisch?)
 - Wie kann das vom Nutzer ergänzt werden? (Bsp: Neue Art erfordert auch direkt eine neue Gattung, Familie etc.)
 - Geographische, politische Struktur mit vielen Ebenen: Europa -> Deutschland -> NRW -> Siegen-Wittgenstein -> Siegen (parent-child relationship)
 - Welche Daten werden zu jedem Objekt gespeichert
 - Felder für: eindeutige Zuordnung, aktuellen Aufbewahrungsort (Regal3, Fach 7), Kommentarfeld
 - Bilder? Z.b. matchen und einbinden aus Bilddatenbank, oder lokal auf dem Server speichern mit Uploadfunktion
 - Welche Vorgänge / Use-Cases gibt es (siehe Background)
 - Datenbank und Website-Design sollte die Use-Cases möglichst einfach und effizient ermöglichen



- IDE
 - Microsoft Visual Studio (ASP.NET, .NET MAUI installieren)
- Programmiersprache
 - C# (ASP.NET Core, .NET MAUI)
- Versionsverwaltung (**zwingend**)
 - Github www.github.com (z.B. privates Repo anlegen)
 - Eigene (**nicht**: Dropbox-Ordner)
 - ➔ Sicherung des Versionsverlaufs, Datenverlust unwahrscheinlich, bessere Zusammenarbeit



- ASP.NET Core – Tutorials und Dokumentation
<https://dotnet.microsoft.com/learn/aspnet>
<https://docs.microsoft.com/en-us/aspnet/core>
- ASP.NET Core Backend für Mobilgeräte
<https://docs.microsoft.com/en-us/aspnet/core/mobile/native-mobile-backend>
- MVC-Muster
https://de.wikipedia.org/wiki/Model_View_Controller



- Folien der Einführungsveranstaltung (this)
 - SQL Server Zugang Anleitung (eigene DB auch möglich)
 - Muster Pflichtenheft
 - Muster Benutzerhandbuch
- zu finden in Mattermost



Zur Teilnahme erforderlich:



- [Formular Ausfüllen](#)



- Mattermost Team [beitreten](#)



Login zu Mattermost über gitlab:
Uni-Mail + Passwort

(JETZT!)



Fragen?

